

SOP 20 — Classify and Dedupe

Camada A.N.I.: Instruments-Tools (classify_and_dedupe.py) · Versão: 1.0 (Protocol 0, 2026-09-09)
 Golden Rule: mudou a lógica → atualiza o SOP → só então o código. · Invariante 1: LLM preenche campos, não decide negócio.

Objetivo

Transformar `RawInput.NewsItem[]` em `ProcessedOutput.NewsCard[]` válidos: o LLM classifica (selo, categoria, unidade) preenchendo um schema; a lógica de negócio (regras de selo, dedupe) é **determinística, no código** — qualquer output de LLM que não valide contra o schema é descartado e vira `needs_review: true`.

Quando roda

- Cron daily build 07:15 sobre o lote da ingest 06:30.
- Cron refresh 12:00 e 18:00 sobre itens novos (cards novos → fila do editor; não publicam direto).

Input

- `RawInput.NewsItem[]` de `.tmp/news/<YYYY-MM-DD>.json` (SOP 10).
- Regras de `gemini.md` (selos, categorias, allowlist, dedupe > 0.82).
- LLM configurado via `LLM_API_KEY` em `.env`.

Output

- `ProcessedOutput.NewsCard[]` → `.tmp/cards/<YYYY-MM-DD>.json` — cada card com `card_id`, `title`, `lede`, `stamp`, `category`, `team_unit`, `source_name`, `source_url`, `needs_review`, `duplicate_of`.
- JSON de relatório: `{ "input_items": n, "cards": n, "duplicates_removed": n, "needs_review": n }`.

Passos determinísticos

1. Ler o lote de `RawInput.NewsItem` da bancada.
2. **Classificação por LLM** (1 chamada por item ou lote pequeno, com schema estrito):
 - Pedir ao LLM **apenas** os campos: `title` (curto, tom rubro-negro, pt-BR), `lede` (1-2 frases, sem copiar texto integral), `stamp`, `category`, `team_unit`.
 - Validar o JSON de resposta contra o schema da CONSTITUTION. Inválido → descartar a resposta e marcar o item com `needs_review: true` (nunca “ajeitar” output inválido).
3. **Regras de selo (código, não LLM):**
 - `oficial` **somente** se a URL for de domínio oficial allowlist (flamengo.com.br e redes oficiais).
 - `rumor` se a fonte não for oficial E o texto contiver “negociando / perto / sondado / exclusividade” **sem confirmação cruzada** (2+ fontes independentes da allowlist com o mesmo fato).
 - Selo ambíguo → `needs_review: true` (não adivinhar).

4. **Dedupe (código):** similaridade de título+entidades > 0.82 entre cards → manter **a mais antiga** e, se houver oficial no grupo, **a oficial**; preencher `duplicate_of` no(s) descartado(s) apontando para o card mantido.
5. Montar `card_id` canônico: `<AAAAMDD>-<categoria>-<nn>` (ex.: `20260909-mercado-01`).
6. Gravar `.tmp/cards/<YYYY-MM-DD>.json` e reportar o resumo.

Edge cases

- LLM fora do ar/timeout → 1 retry; persistindo, **degradação determinística**: selo/categoria por heurística simples de palavras-chave com `needs_review: true` (nunca inventar classificação confiável sem LLM validado).
- Output do LLM fora do schema (campo extra, enum inválido, JSON quebrado) → descartado; item → `needs_review`.
- Conflito oficial × mais antiga no dedupe → **vence a oficial** (regra do blueprint); a mais antiga não-oficial entra como `duplicate_of` da oficial.
- Texto contém gatilho de rumor mas há confirmação cruzada (2+ fontes allowlist) → não é rumor automático; deixar `confirmado` somente se as fontes forem independentes; dúvida → `needs_review`.
- Item de fonte fora da allowlist → nem chega aqui (SOP 10 já filtra); se chegar, descartar com log.

Rate limits / limites conhecidos

- Respeitar RPM/token do provedor LLM escolhido (fixar na Phase L); batches pequenos com pausa se necessário.
- Custo: 1 classificação por item novo; itens duplicados por URL já vistos não são reclassificados (SOP 10).

Testes

- Fixture: `fixtures/news_bundle.json` — 8 itens (2 duplicados entre si, 1 oficial, 1 rumor). A tool deve: manter o par duplicado com 1 card (`duplicate_of` preenchido), selar `oficial` o item de domínio `flamengo.com.br`, selar `rumor` o item com gatilho sem confirmação e marcar `needs_review` quando faltar contexto.